4. A service centre uses a fixed-size request buffer in which released positions must be reused. Write a C program to implement a Circular Queue using an array with insertion, deletion, display, overflow and underflow operations. Demonstrate that positions freed after deletion can be reused for new requests.

Source code :-

```
#include <stdio.h>

#define SIZE 5

int queue[SIZE];

int front = -1, rear = -1;

void enqueue(){
   int value;
   if ((rear + 1) % SIZE == front) {
      printf("Queue Overflow\n");
      return;
   }
```

```
    printf("Enter request: ");
    scanf("%d", &value);
    if (front == -1)
        front = rear = 0;
    else
        rear = (rear + 1) % SIZE;
    queue[rear] = value;
    printf("Request inserted\n");
}

void dequeue()
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return;
    }
    printf("Deleted request: %d\n", queue[front]);
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % SIZE;
}
void display()
{
    int i;
    if (front == -1)
    {
        printf("Queue is empty\n");
        return;
    }
```

```
    printf("Queue: ");
    i = front;
    while (1)
    {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}
int main()
{
    int choice;
    while (1)
    {
        printf("\n--- Circular Queue ---\n");
        printf("1. Insert\n");
        printf("2. Delete\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
```

```
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice\n");
      }
   }
   return 0;
}
```

Output :-

```
--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter request: 5
Request inserted

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter request: 0
Request inserted

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter request: 5
Request inserted
```

```

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue: 5 0 5

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2
Deleted request: 5

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue: 0 5
```